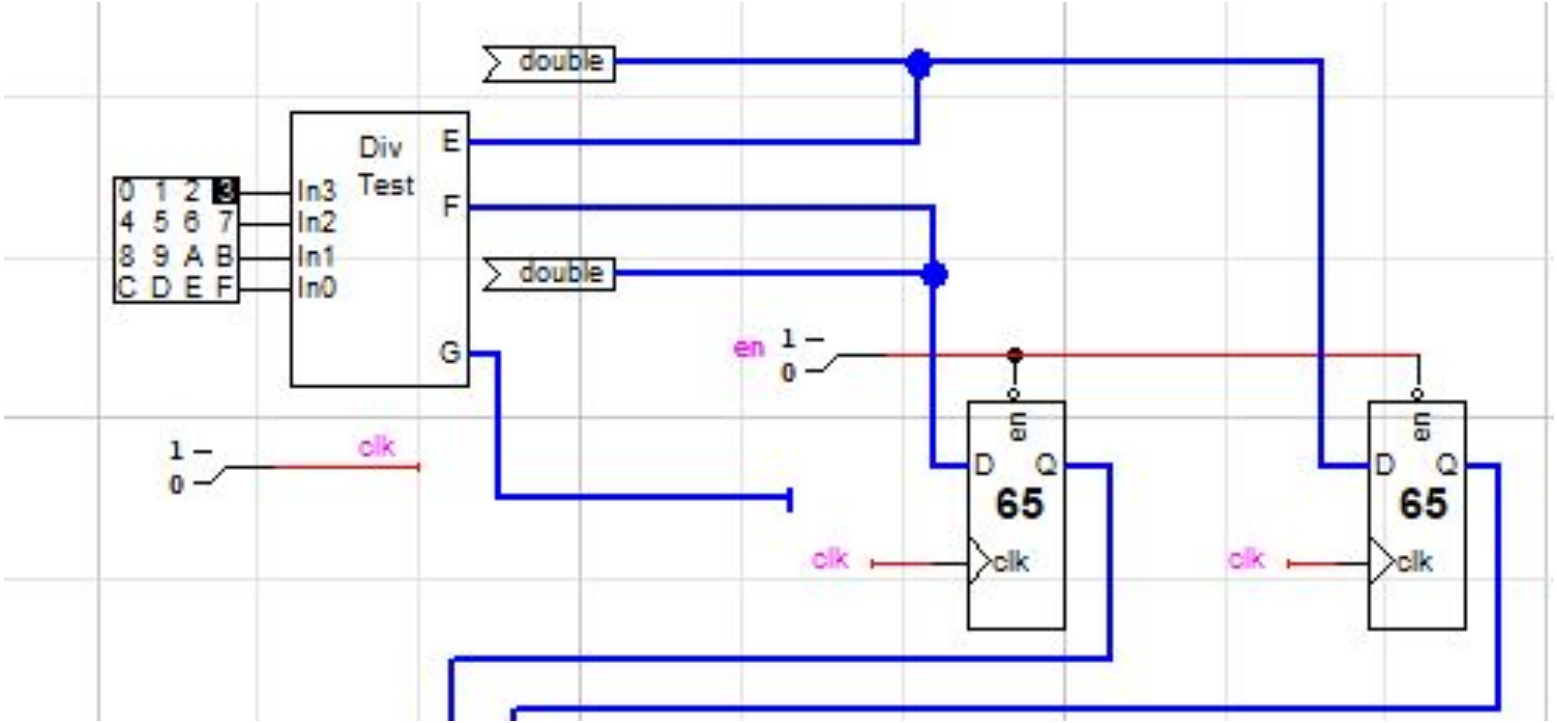
Divider Exponent

Nahom Solomon
Jaskaran Mann

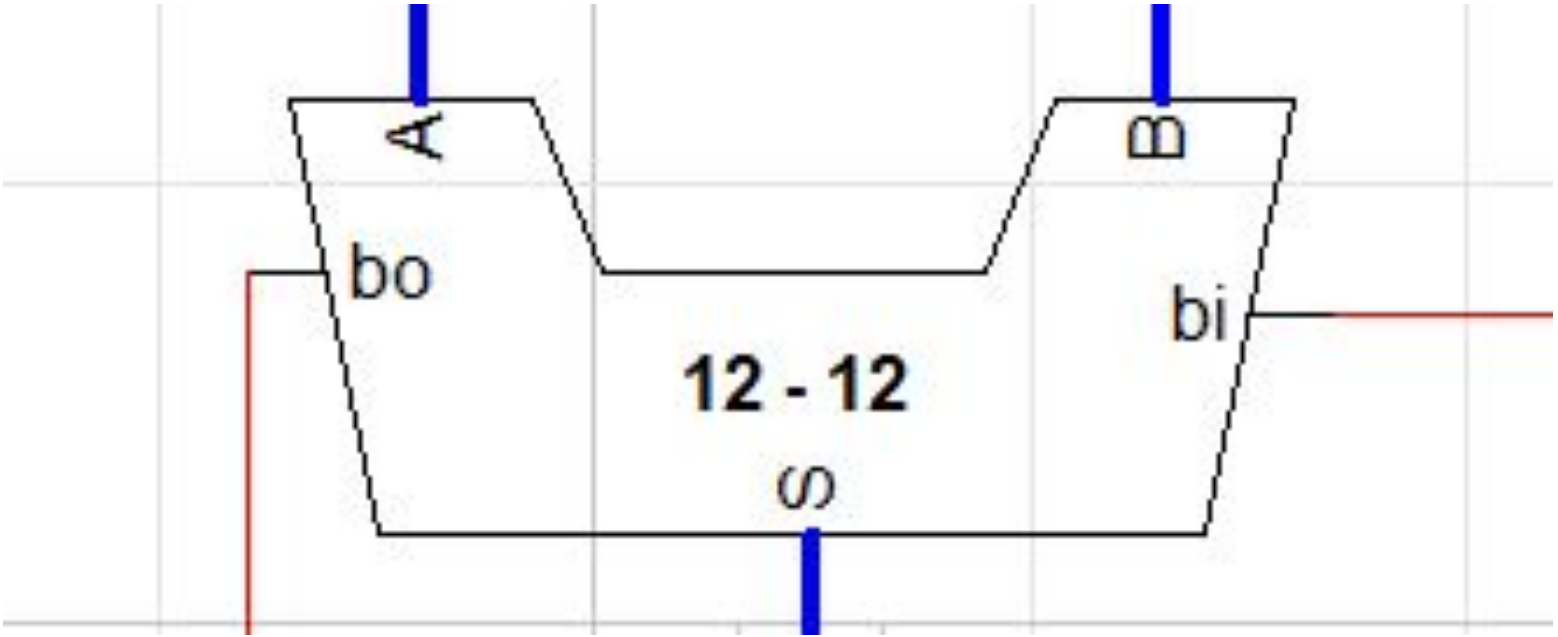
Stage 1: Input Loading



Stage 1: Input Loading

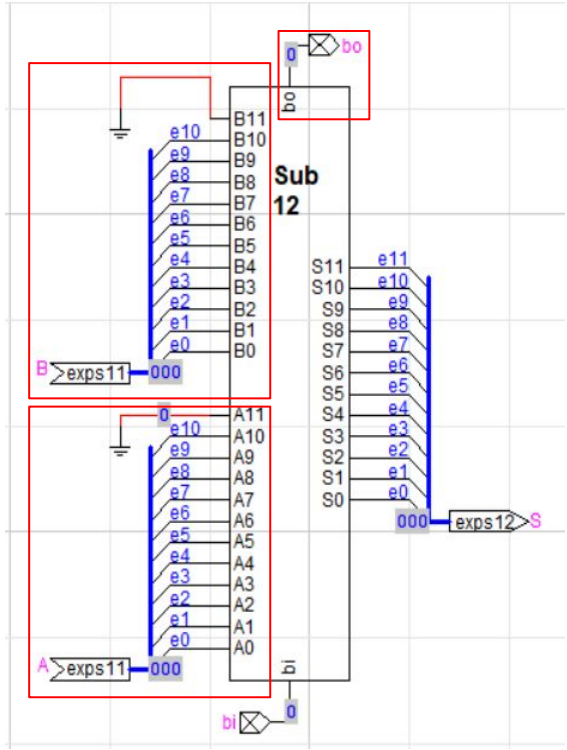
- The Divider Test allows us to select up to 16 test inputs with expected results to validate our circuits is performing the correct operations
- Inputs are display in the order of
  - 1 Hidden bit (m52)
  - 1 sign bit (sign)
  - 11 exponent bits (e10..e0)
  - 52 mantissa bits (m51..m0)
  - Total number of bits = 65 bits
- The inputs represent 64- bit floating- point numbers E (numerator), F (denominator), and G (expected results)
- E and F go into two 65-bit registers to capture and hold the values so they don't change during computation. They prepare the fields for the exponent processing steps.

Stage 2a: Exponent Subtraction



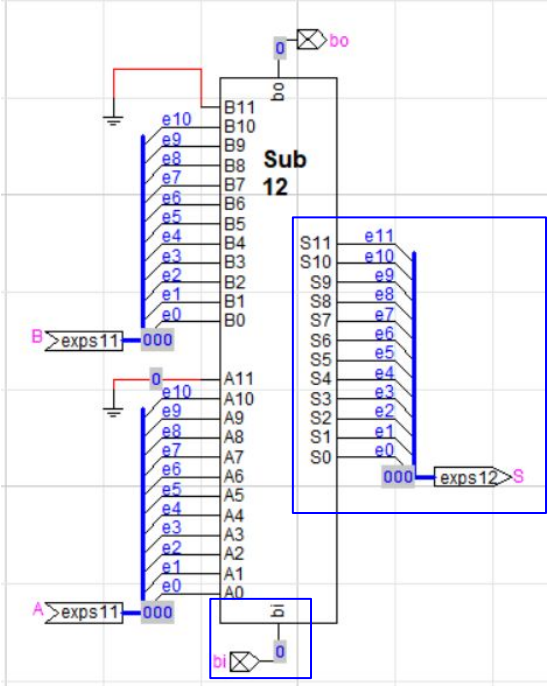
Stage 2a: Exponent Subtraction

- The exponent from B is subtracted from A. This is needed for floating-point division to compute the new exponent value
  - Result Exponent = A_exp - B_exp
- Inputs are A[11:0], B[11:0], and bi
  - A[11] and B[11] = 0
    - Grounded to force sign bit to be a positive input
  - A[10:0] and B[10:0] = 11-bit exponent
    - e10..e0
  - bi (bit in) controls what operation is being performed, which comes from the inverted value of the most significant mantissa bit (m55)
    - bi = 1
      - Performs two's complement subtraction (A - B - 1)
        - Two's complement: [A+(~B+1)]
    - bi = 0
      - Performs basic subtraction (A - B)

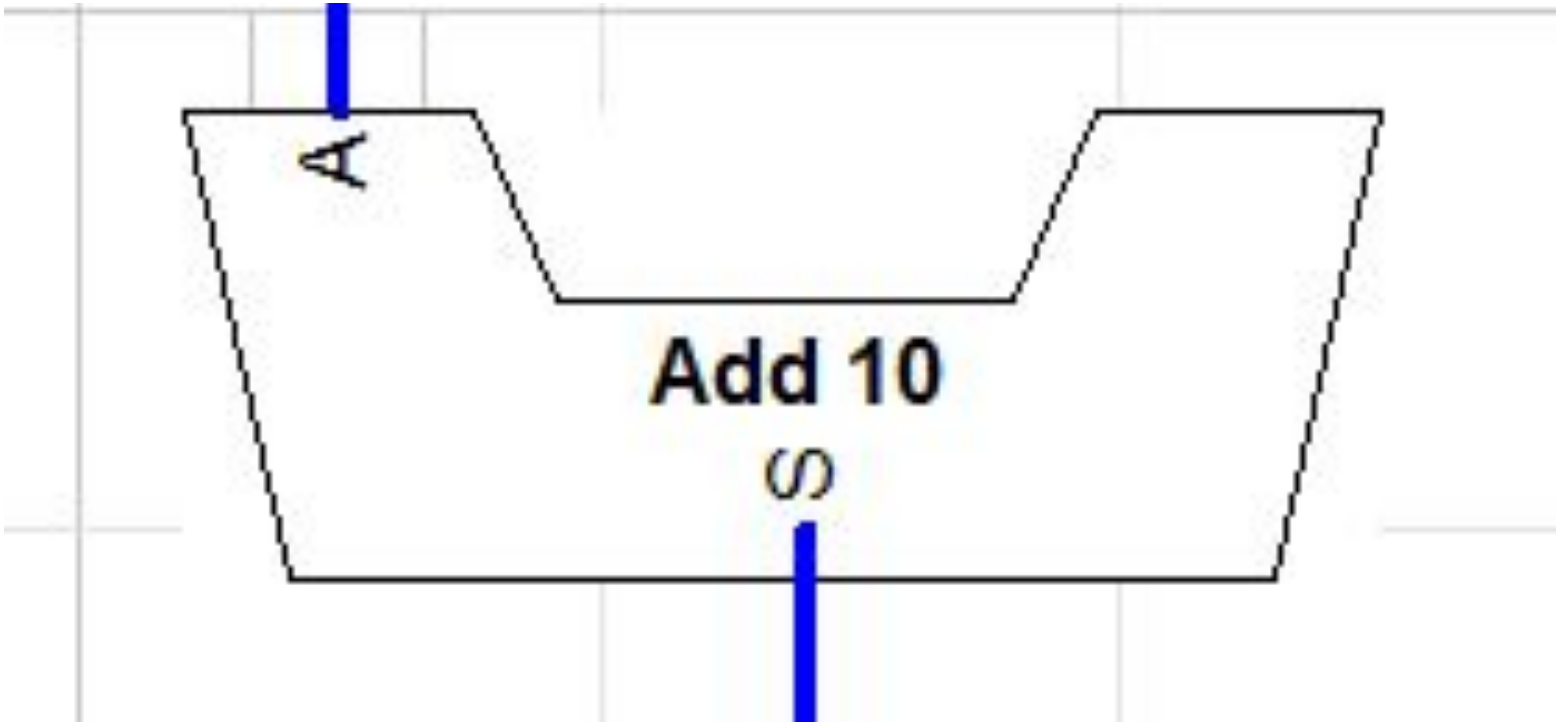


Stage 2a: Exponent Subtraction

- Outputs are S and bo
  - S (sum) represents the 12-bit signed result
    - The exponent difference
      - Which can be positive or negative, in two's complement form
    - e11..e0
  - bo (bit out) is the borrow-out flag, which provides comparison feedback without modifying the result
    - bo = 1
      - Borrow happened (A < B)
    - bo = 0
      - no Borrow happened (A >= B)

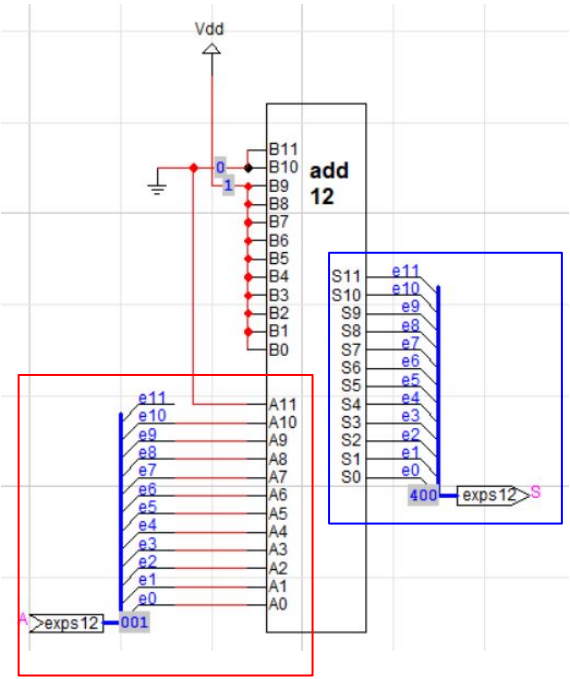


Stage 2b: Bias Addition

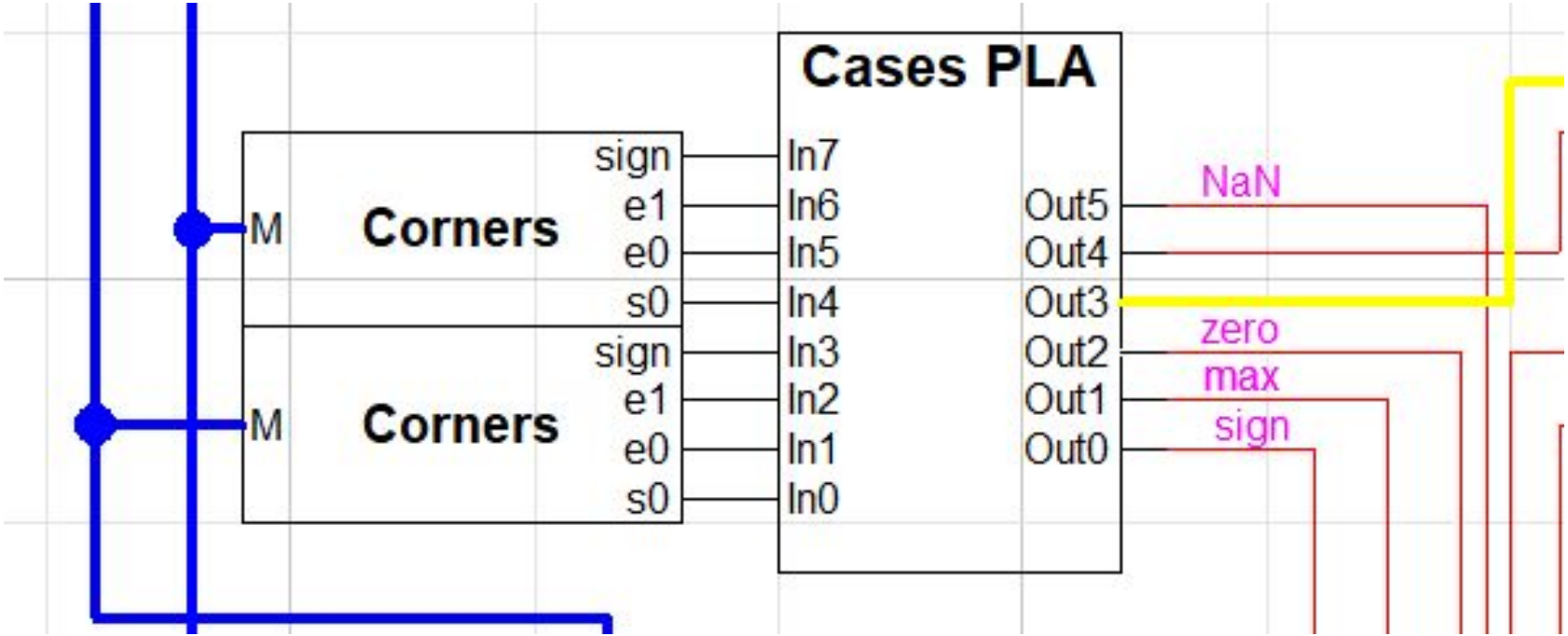


Stage 2b: Bias Addition

- After subtracting the exponents in Stage 2a, the result is unbiased
  - Means it does not match the IEEE-754 floating- point format, so we need to add 1023 (decimal)
    - Result Exponent (biased) = Result Exponent (unbiased) + 1023 (decimal)
  - Inputs are A[11:0], the result exponent (unbiased) from the subtractor
    - e11..e0
  - To add 1023 (decimal), we hardcode
    - B[11:10] = 0
    - B[9:0] = 1
  - Output is S, which is the 12- bit exponent (biased)



Stage 3: Exception Detection (PLA)



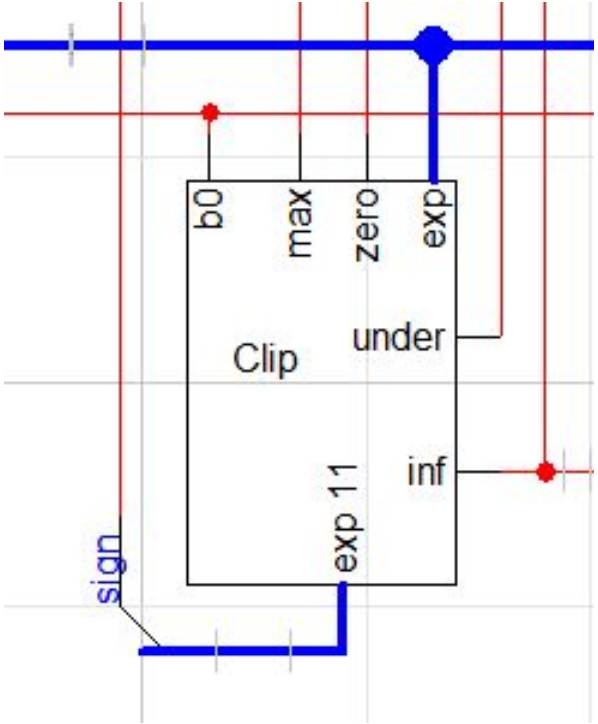
Stage 3: Exception Detection (PLA)

- We detect special floating-point corner cases by examining the signs, exponents, and significands of the inputs A and B
- Which generates four key signals with will be the inputs into the PLA
  - sign
    - identifies the sign bit
  - e1
    - Identifies if the exponents field is all 1’s
      - Special cases: Infinity or NaN
  - e0
    - Identifies if the exponent field is all 0’s
      - Special cases: Zero or Subnormal
  - s0
    - Identifies if the significand field is all 0’s

Stage 3: Exception Detection (PLA)

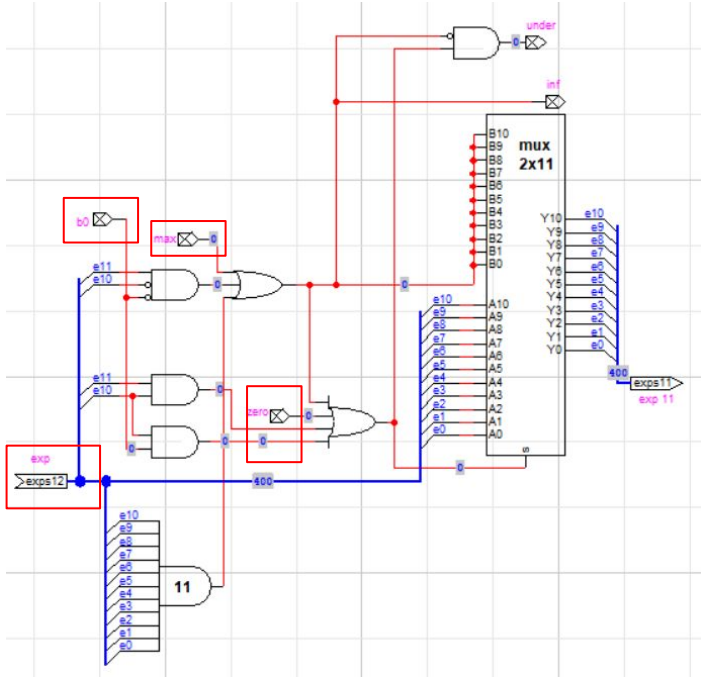
- The PLA is configure to take the inputs from the corner cases and flag conditions thats have been detected
  - NaN
    - Not a Number detected
      - X100XXXX
      - XXXXX100
      - X101X101
      - XX11XX11
  - div0
    - Division-by-zero detected (denominator)
      - X00XXX11
      - X010XX11
      - X101XX11
      - XX11XX11
  - invalid
    - Invalid operation (example 0/0, inf/inf)
      - X00XX010
      - XX10X010
  - zero
    - zero detected (numerator)
      - X011X1X1
      - X011X00X
  - max
    - maximum finite value detected
      - X1X1XXXX
      - X1X0XXXX
      - XXXXX100
      - X00XX011
      - X01XX011
  - sign
    - final result sign bit detected
      - 0XXX1XXX
      - 1XXX0XXX
      - XX11XX11

Stage 4: Clip Logic



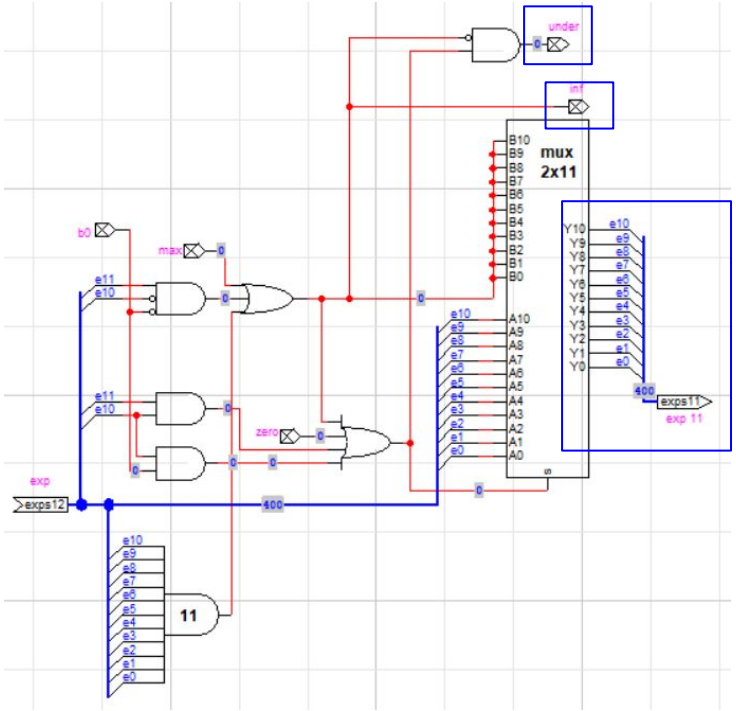
Stage 4: Clip Logic

- The clip logic is needed to make sure the exponent is valid after subtraction and biasing. Without the clip logic is could output the wrong exponent when the results is too small (underflow) or too big (infinity)
  - Inputs
    - bo
      - from subtractor (Stage 2a)
    - Max
      - PLA (Stage 3)
    - Zero
      - PLA (Stage 4)
    - Exp
      - From adder (Stage 2b)

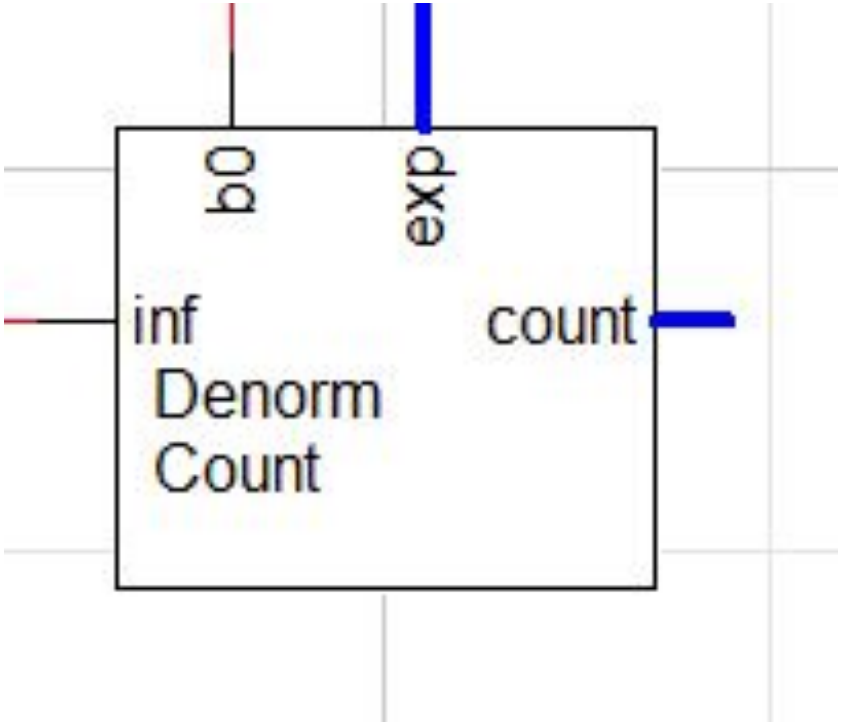


Stage 4: Clip Logic

- The output of the Clip is the adjusted exponent, the underflow and infinity flags, and then the sign bit is added back to prepare for the next stage.
  - Underflow
    - Detected when the exponent is too small to represent normally
  - Infinity
    - Detected when the exponent is too large to represent normally
  - Exponent
    - After the Clip Logic, the sign bit is added to the exponent output to complete the floating-point result
  - The Clip ensures the final exponent is valid by limiting (or “clipping”) its value to stay within the representable range.

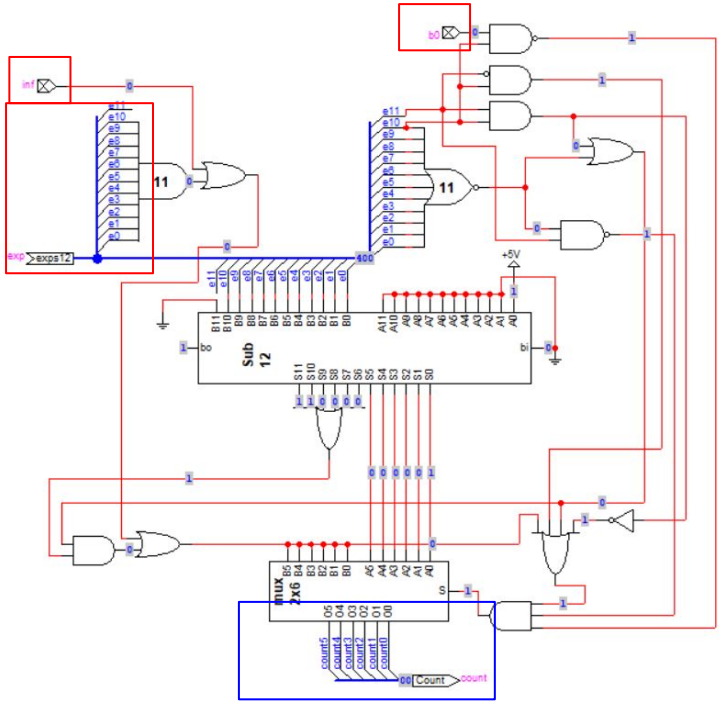


Stage 5: Denormalization

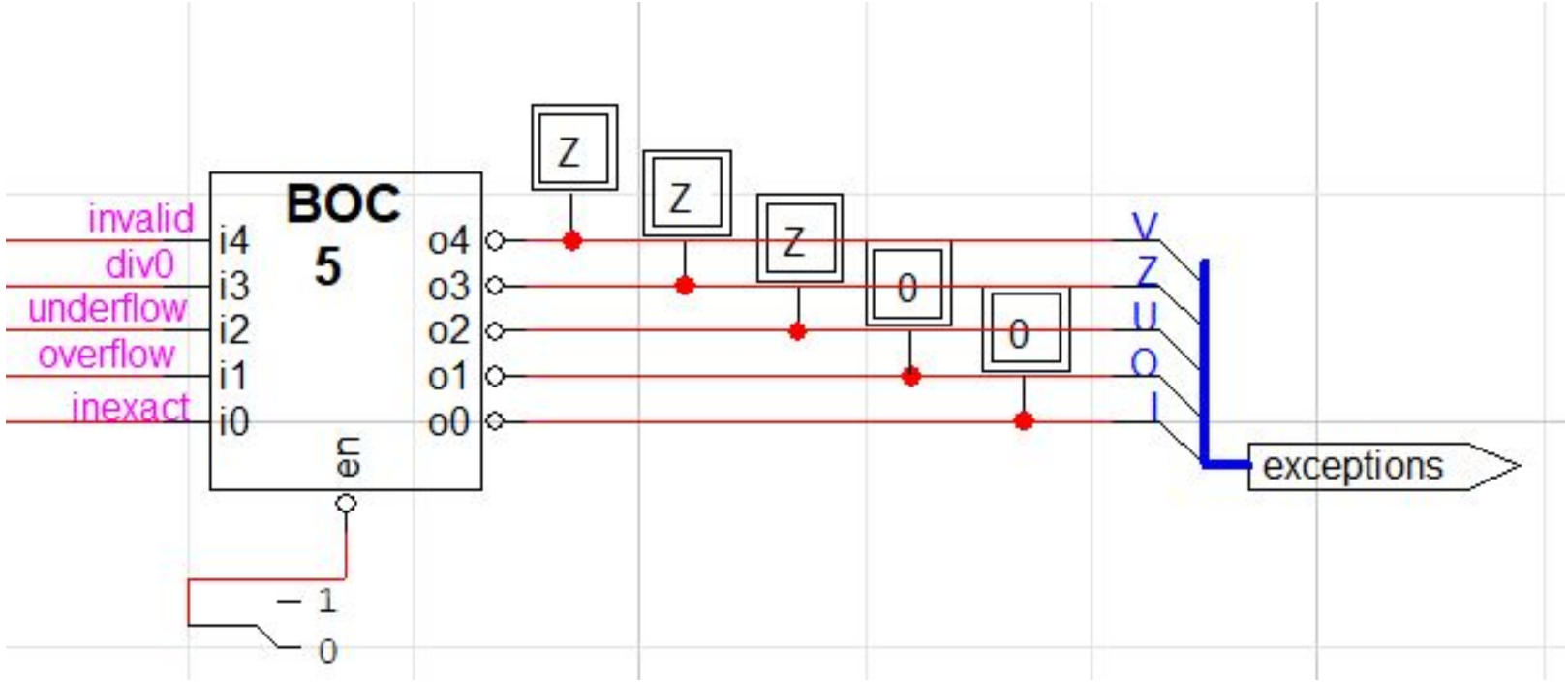


Stage 5: Denormalization

- Denormalization lets the results that are too small still be represented by forcing the exponent to zero. It prevents losing tiny numbers completely
- Inputs
  - inf
    - Informs us the result should be maximum (Stage 4 - Clip)
      - Denormalization will not happen
  - bo
    - Informs us if underflow happened (Stage 2a - Exponent Subtraction)
      - Denormalization will happen
  - exp
    - From the adder (Stage 2b - Bias Addition)
- Output
  - Count
    - Represents the adjusted exponent value after clipping (Stage 4) and denormalization
    - The output count is 6 bits because our divider only needs to represent smaller exponents, and the extra bits from math are no longer needed
    - The purpose of the Count is to represent how much the mantissa needs to be right-shifted during denormalization when the exponent is too small to be represented normally.

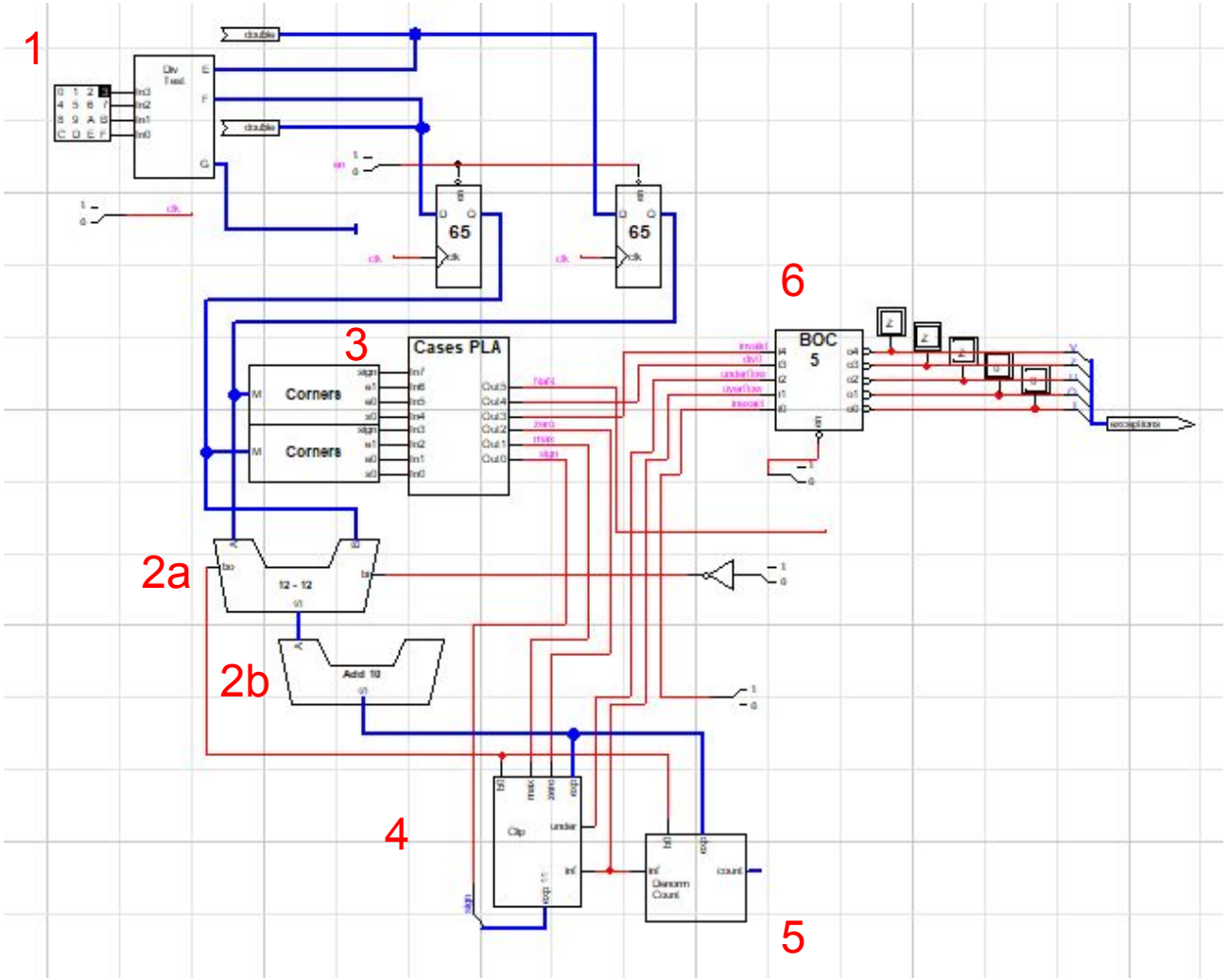


Stage 6: Exceptions



Stage 6: Exceptions

- The flags are collected into an open collected buffer to hold and export the final exception signals
  - Inputs/Outputs
    - Invalid (V)
      - Detected from PLA (Stage 3)
    - Div0 (Z)
      - Detected from PLA (Stage 3)
    - Overflow (O)
      - Detected from inf flag from Clip (Stage 4)
    - Underflow (U)
      - Detected from Clip (Stage 4)
    - Inexact (I)
      - Detected from Round PLA and Denormalization from Mantissa
  - If an exception is flagged, it will be collected as a zero (0). If no exception is flagged and the result is not zero, it will remain as Z.


1
Div Test
double
double
65
65
clk
3
Cases PLA
Corners
Corners
6
BOC 5
2a
12 - 12
2b
Add 10
4
Clip
5
Denorm Count

Divider Mantissa

Seth Iris Canonigo
Sanjay Rangarajesh

Background

We are using a Fast Division Algorithm using Newton's Method

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Goal is to find $x = \frac{1}{D}$ when $f(x) = 0$

So that $\boxed{x \times N = Quotient}$

Finding x

Given: $f(x) = \frac{1}{x} - D$ and $f'(x) = -\frac{1}{x^2}$

Then:

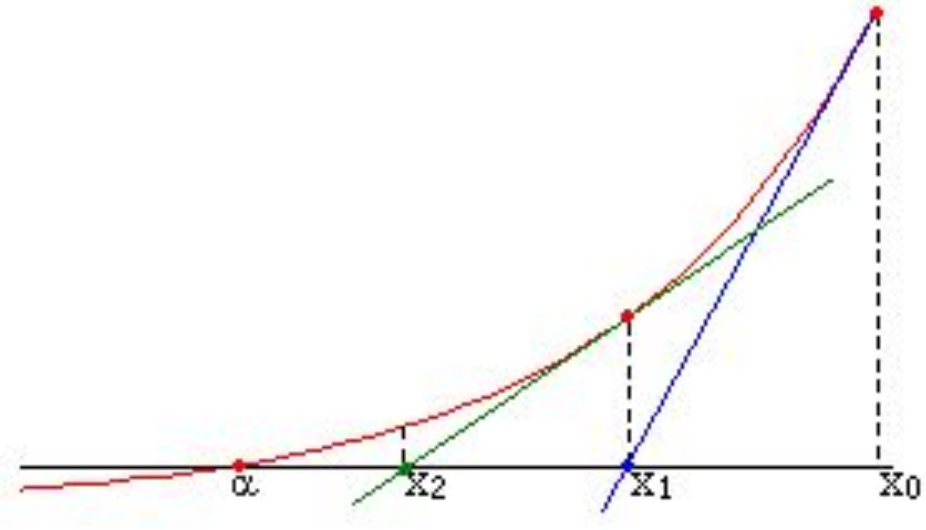
$$x_{n+1} = x_n - \frac{\frac{1}{x_n} - D}{-\frac{1}{x_n^2}} = x_n(2 - x_n \times D)$$

$$\boxed{x_{n+1} = x_n(2 - x_n D)}$$

Newton’s Method

Using Newton’s Method, each successive iterations of the roots of f(x), which is 1/D, becomes more and more accurate and precise.

For dividing mantissa, you only need **three** iterations, to get a double-precision floating-point number



Finding x_0

$$x_0 = \frac{1}{2}\left[\frac{d_{round}((D-1)2^M)}{2^B} + 1\right]$$

where M = 7 and B = 8

There are two methods to finding d(n).

<u>Linear Approximation method: **(Hard Way)**</u>

$$d_{round}(n) = 2^B\left[\left[\frac{2}{1+\frac{n}{2^M}}\right] - 1\right]$$

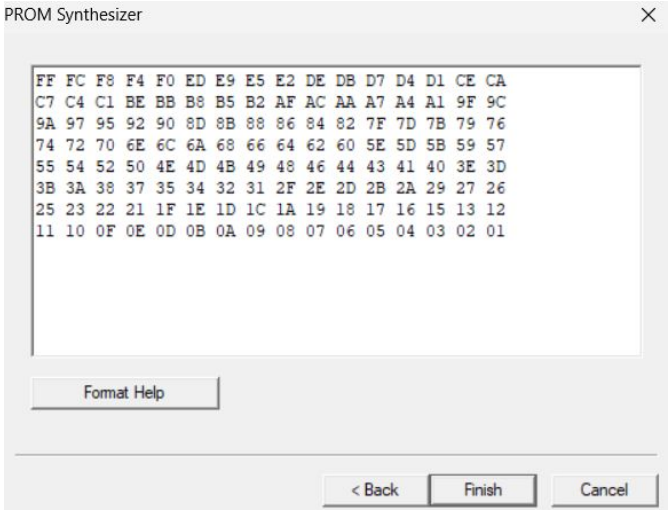
Finding x_0

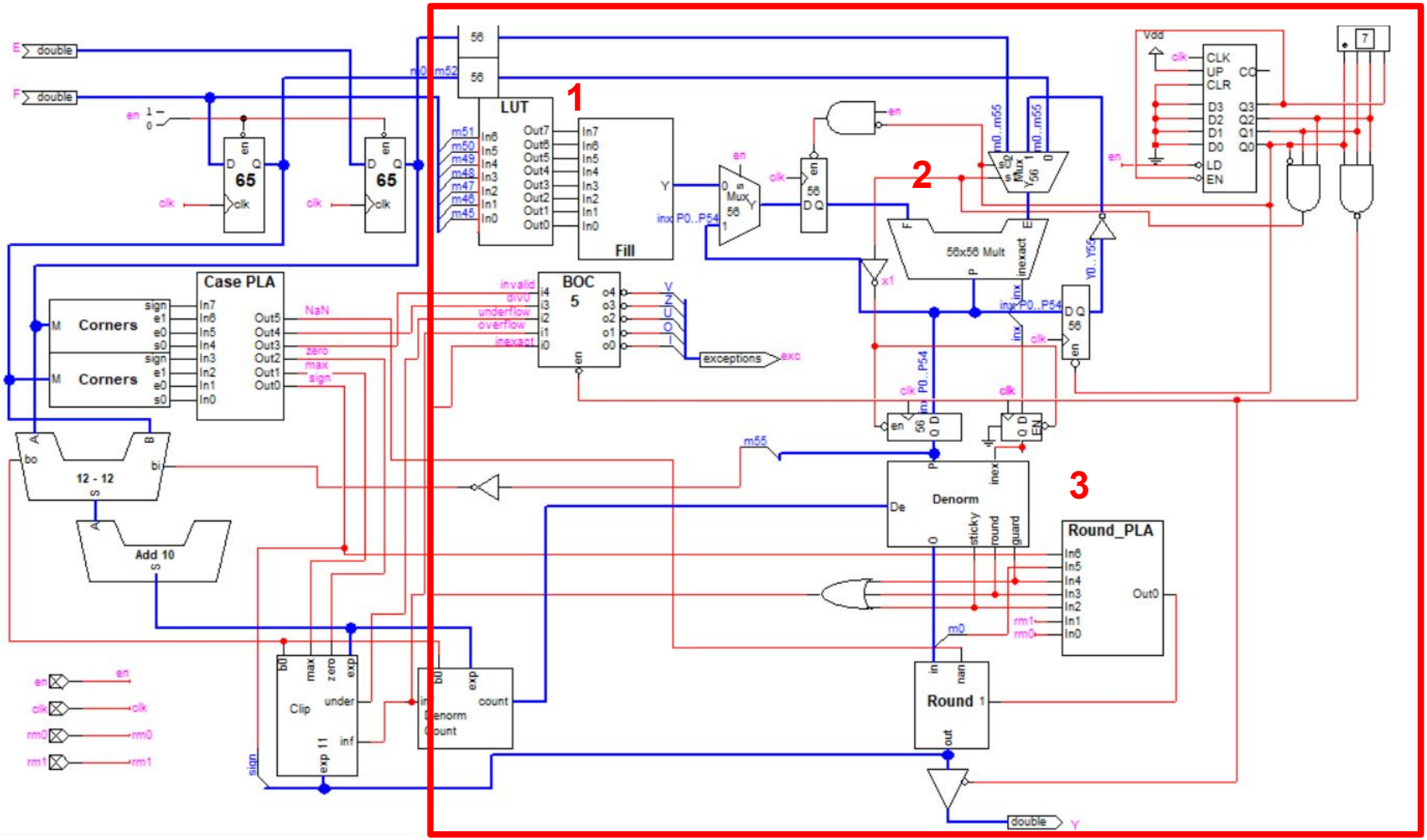
<u>Look-Up Table method: **(Easy Way)**</u>

The easy way is to make a PROM based on the first seven bits of the significand

Where M is the number of inputs and B is the number of bits

$$d_{round}(n) =$$

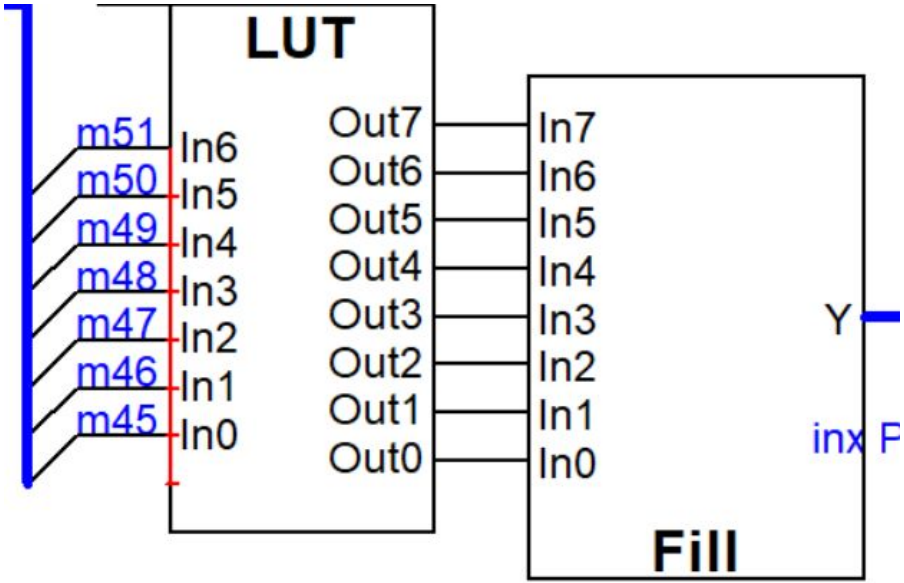



E
double
F
double
en
65
clk
LUT
Fill
1
2
3
Case PLA
Corners
Corners
12 - 12
Add 10
BOC 5
invalid
divu
underflow
overflow
inexact
exceptions
exc
NaN
zero
max
sign
56x56 Mult
Mux 56
Denorm
Round_PLA
Round 1
Clip
Denorm Count
count
Vdd
CLK
UP
CLR
D3
D2
D1
D0
LD
EN
Q3
Q2
Q1
Q0
CO
m0..m55
P0..P54
Y0..Y55
m55
rm1
rm0
Y

Part 1: Lookup Table

As mentioned before, the look-up table is a PROM with set 8-bit outputs d(n) based on the first seven bits of the significand. In this case, the seven most significant bits of the 53-bit mantissa (sans hidden bit).

d(n) will then become the inputs for the Fill block.



Remember:

$$x_0 = \frac{1}{2}\left[\frac{d_{round}((D-1)2^M)}{2^B} + 1\right]$$

M = 7, B = 8

Therefore the 8 bits for d(n) will have to be shifted right eight times, add 1 bit, then shift right one more time

Part 1: Fill

Essentially, we are normalizing the integer d(n) and also push the hidden bit as well.

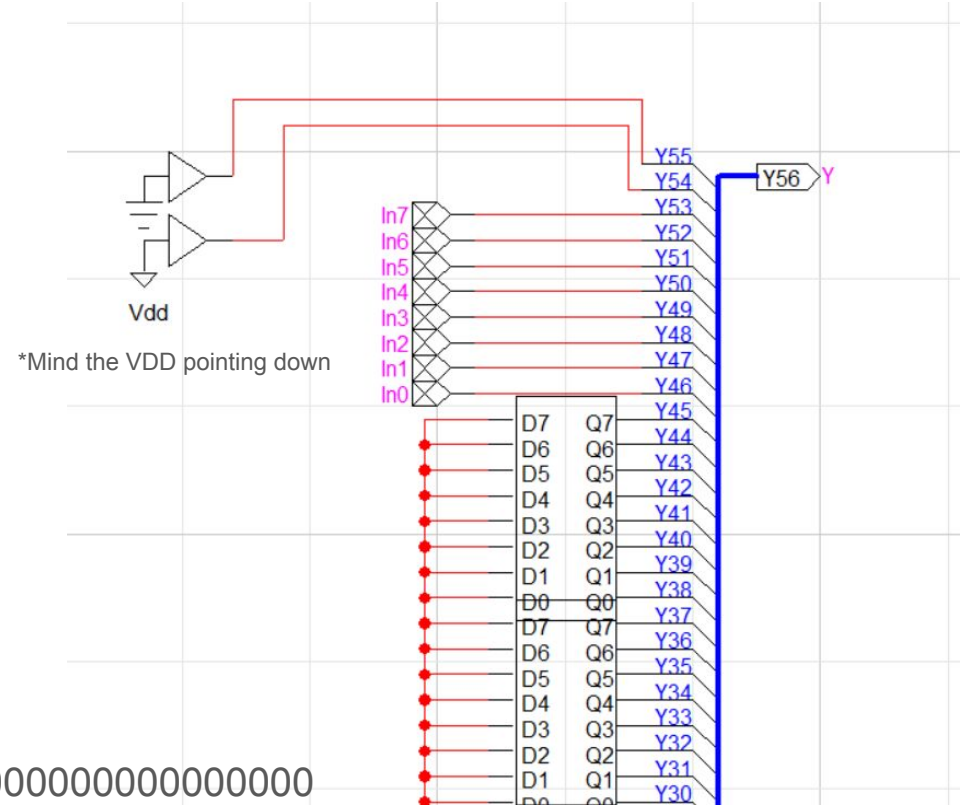
XXXXXXXX.0

0.XXXXXXXX

1.XXXXXXXX

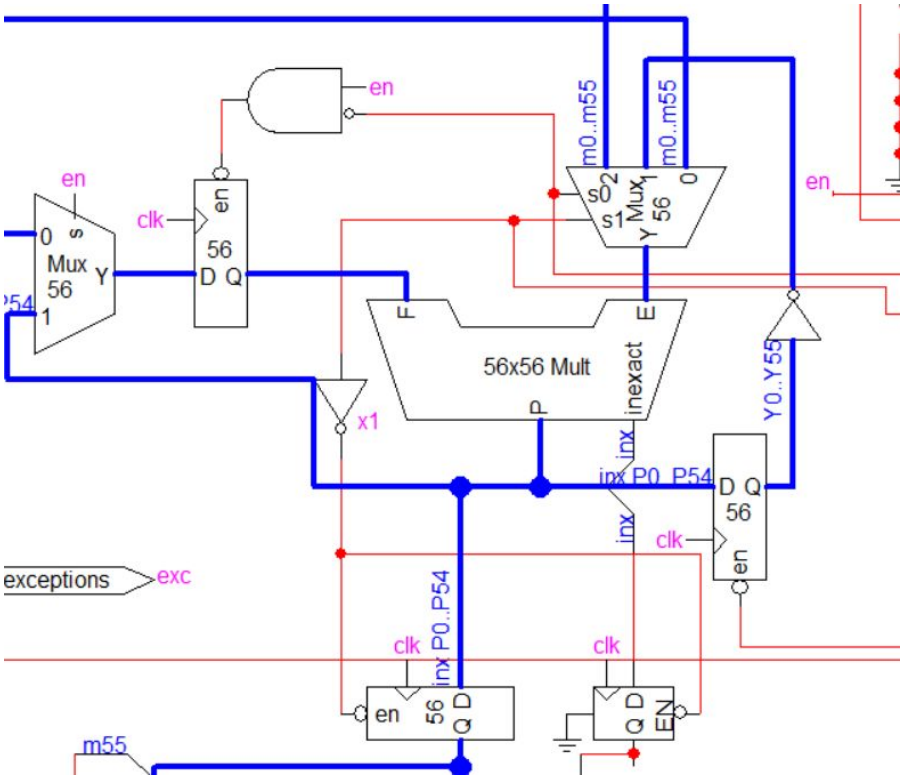
0.1XXXXXXXX

So that:
01XXXXXXXX0000000000000000000000000000000000000000



Part 2: Newton's Method Iterations

Now that we have x_0 ,we can begin using Newton's Method to find a more precise approximation of 1/D.



Remember:

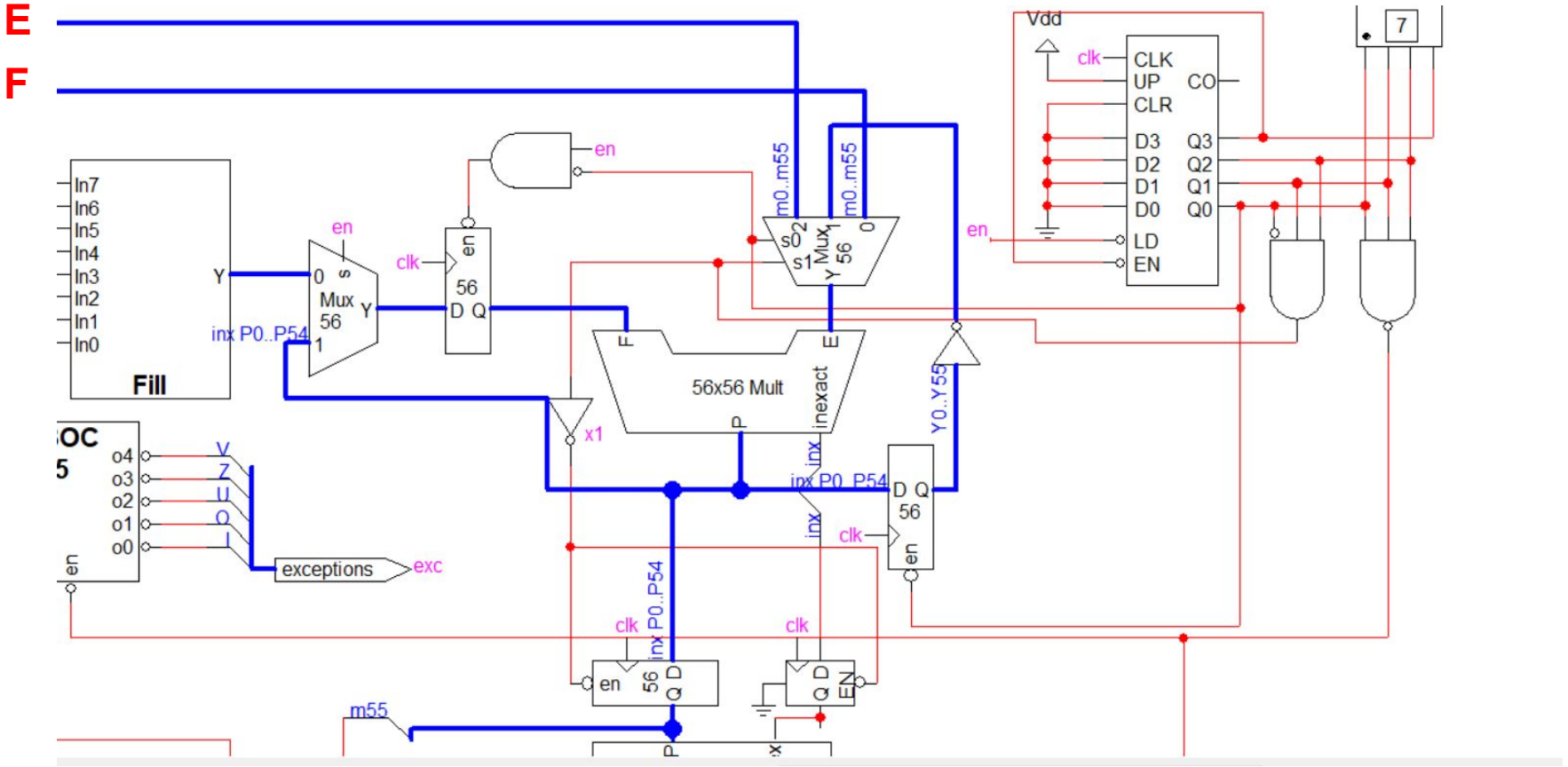
$$x_{n+1} = x_n(2 - x_n D)$$

Therefore, you need to get the current value of x and multiply it with the denominator. Then, you need only need to find one's complement of that value and multiply it by the current value of x to get the next value of x.

And repeat that two more times

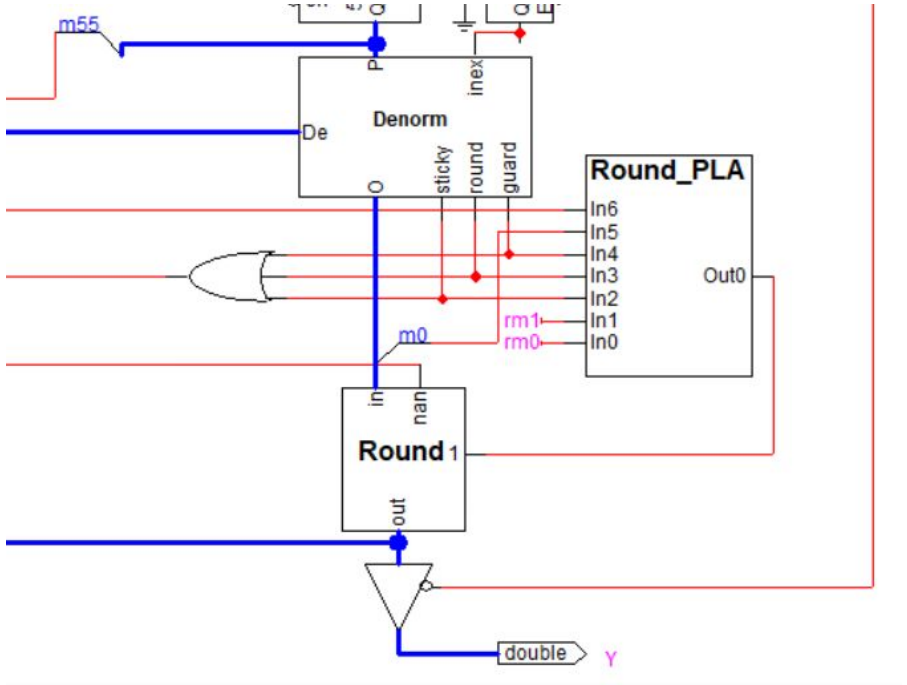
And finally, multiply the reciprocal with the numerator to get the quotient.

Part 2: Newton's Method Iterations



Part 3: Denormalizing and Rounding

The denormalizer shifts the mantissa based on the Denorm count to correctly handle small results. The 55 bit mantissa + hidden bit is converted to a 52 bit mantissa + hidden bit. The bottom 3 bits are taken out of the original mantissa and go to the Round PLA as Guard, Round and Sticky signals.



Quick summary on rounding

Guard Bit (G): The first bit beyond the mantissa to aid in rounding.

Round Bit (R): The second bit beyond the mantissa to refine rounding decisions.

Sticky Bit (S): Tracks any 1s beyond the round bit, ensuring that any remaining bits are not ignored in rounding.

GRS Bits	Action (Round-to-Nearest, Ties-to-Even)
000	No rounding, keep the mantissa unchanged
001	No rounding, keep the mantissa unchanged
010	No rounding, keep the mantissa unchanged
011	Round up
100	Round up if LSB is 1 (ties to even)
101	Round up
110	Round up
111	Round up

More rounding

IEEE 754 has four rounding modes:

Modes 00 (Round to Nearest/Ties to Even): Apply round up based on even/odd LSB and Ground Round Sticky bits.

Modes 01 (Truncate): No rounding, just discard the extra bits.

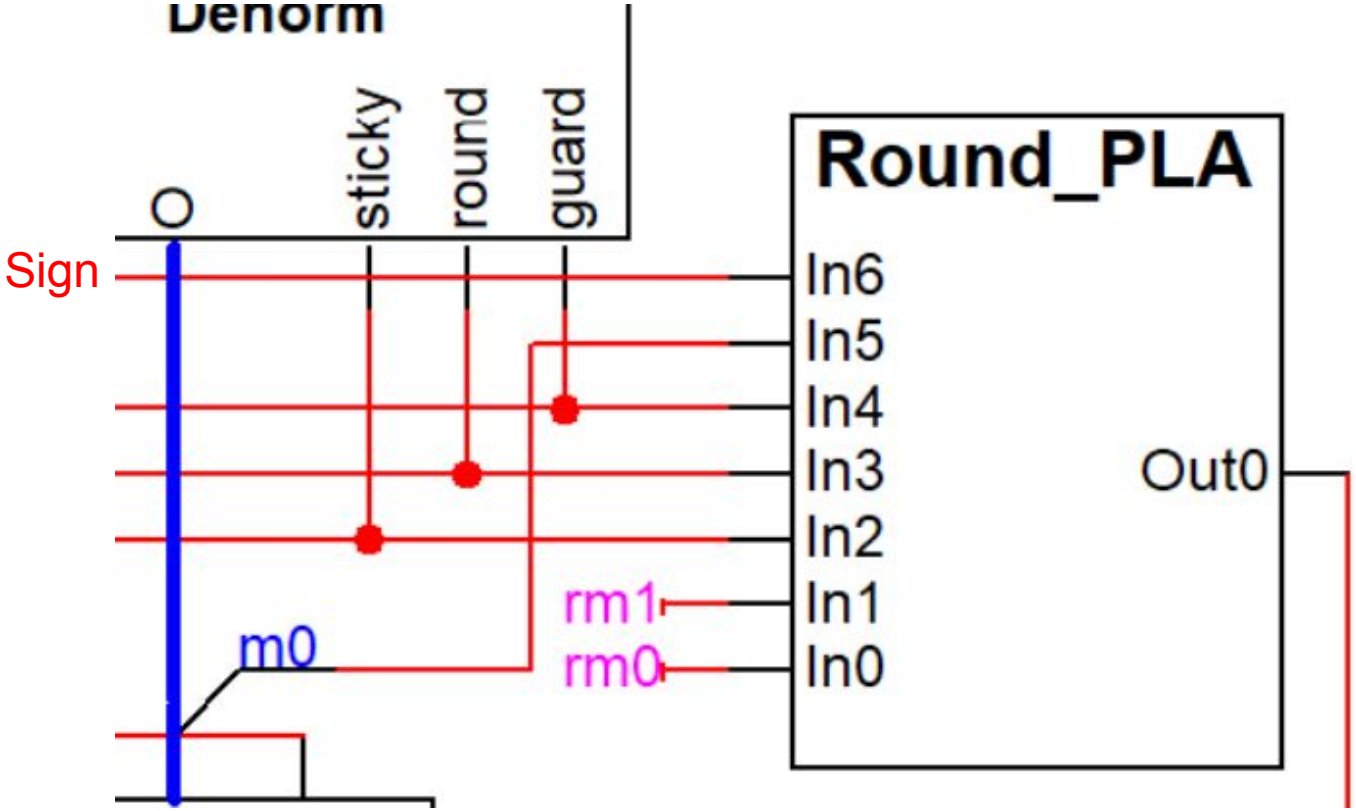
Modes 10 (Ceiling): Round up if positive, truncate if negative.

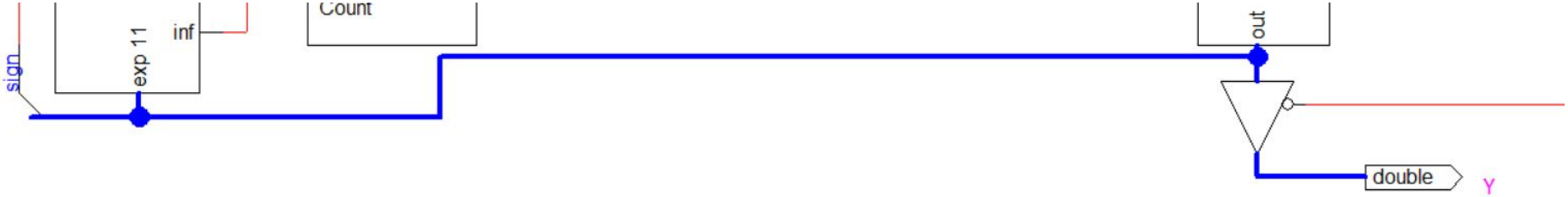
Modes 11 (Floor): Round down if negative, truncate if positive.

Since we can only output 53 bits. We will have to round the 3 extra bits from the 56-bit quotient.

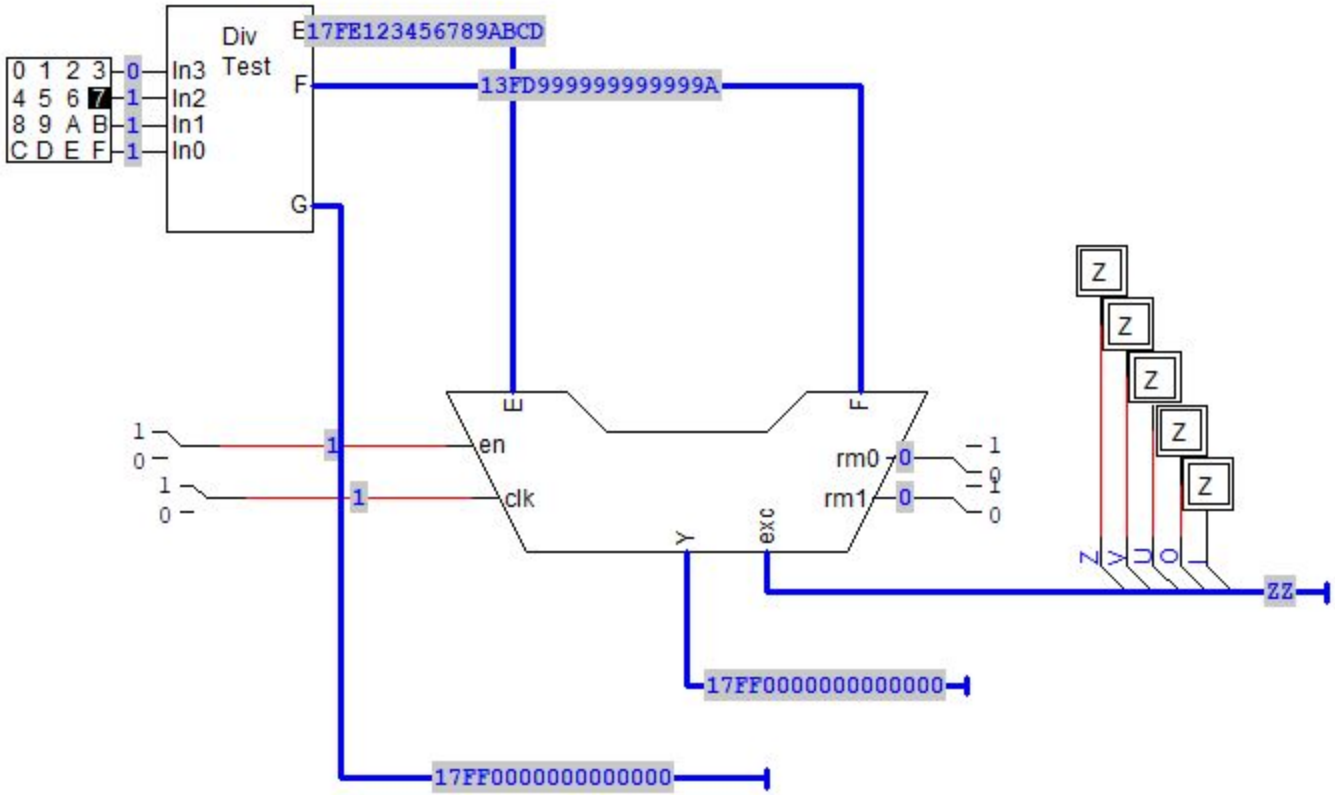
Round PLA

X110000
XX11X00
XX1X100
0X1XX10
0XX1X10
0XXX110
1X1XX11
1XX1X11





Hidden bit | Sign | 11-bit exponent | 52-bit mantissa


0 1 2 3
4 5 6 7
8 9 A B
C D E F
0
1
1
1
In3
In2
In1
In0
Div
Test
E
F
G
17FE123456789ABCD
13FD999999999999A
17FF0000000000000
17FF0000000000000
1
0
1
0
1
1
en
clk
E
F
rm0
rm1
0
0
1
0
1
0
Y
exc
Z
Z
Z
Z
Z
Z
Y
C
O
ZZ

Cases

- Case 0
  - num: 3.0
  - denom: 1.375
  - results: 2.1818181818181817
- Case 1
  - num: 1.0
  - denom: 0.0
  - results: inf
- Case 2
  - num: 1.9999999999999998
  - denom: 1.0000000000000002
  - results: 1.9999999999999993
- Case 3
  - num: inf
  - denom: 4.9406564584124654e-324
  - results: inf
- Case 4
  - num: 0.0
  - denom: 0.0
  - results: 0,0xfff80000 # nan
- Case 5
  - num: 100.0
  - denom: 10.0
  - results: 10.0
- Case 6
  - num: 9.6276454555959567e+307
  - denom: 0.59999999999999998
  - results: 1.6046075759326596e+308
- Case 7
  - num: 9.6276454555959567e+307
  - denom: 0.40000000000000002
  - results: inf
- Case 8
  - num: 1.2345e-307
  - denom: 2.0
  - results: 6.1725e-308
- Case 9
  - num: 1.2345e-307
  - denom: 10.0
  - results: 1.2345000000000002e-308
- Case A
  - num: 4.9999999999999995e-308
  - denom: 10000000000000000
  - results: 4.9406564584124654e-324
- Case B
  - num: -1000000
  - denom: 3.0
  - results: -333333.33333333331
- Case C
  - num: -9.9999999999999995e-07
  - denom: 3.0
  - results: -3.3333333333333333e-07
- Case D
  - num: 0.4
  - denom: -0.1
  - results: -4
- Case E
  - num: -27.0
  - denom: -9.0
  - results: 3.0
- Case F
  - num: inf
  - denom: 3.0
  - results: inf